MLOps Portfolio: Tools and Concepts Primer

Updated 29 October 2026. Learn these concepts in order, then connect each one to a file and an observed result. This project uses Python and infrastructure configuration, with no JavaScript frontend.

1. MLOps and the difference between a model and a system

A model maps inputs to predictions. A working system must also load the right artifact, accept valid inputs, survive failures and report whether it is ready. MLOps applies software and operational practices to the model lifecycle. This repository demonstrates some of those practices; it does not implement every stage of production MLOps.

> Analogy: a recipe is the model. A restaurant also needs a kitchen, orders, reliable equipment and a way to notice failures. A good recipe cannot compensate for lost orders or an unavailable oven.

Study order: understand the 40-message fixture and classifier, run Python serving, follow one Kafka record, inspect the Kubernetes probes, then use Terraform against LocalStack. Treat Bedrock as an optional, unverified live integration.

2. Python modules, environments and dependency versions

A module is a Python source file. A virtual environment gives one project its own installed packages. The serving image uses Python 3.11; CI also checks Python 3.12. Install the requirements for the modules you run. Test and PDF dependencies are separate from the serving requirements.

The audit found that `kafka-python==2.0.2` could not import on Python 3.12. The repaired requirements use 2.2.21. Model training also pins NumPy and SciPy alongside scikit-learn. A dependency freeze in the evidence records the actual resolved environment; selected version pins do not lock every transitive package or Docker base-image byte.

> Analogy: each environment is a separate toolbox. Writing down which tools you used makes a repair repeatable. Merely saying "use a toolbox" does not identify compatible tool sizes.

3. Classification, labels and the fixture

Classification chooses a category. Here the labels are `ham` and `spam`, with 20 examples of each. Forty curated messages are useful for running an operations demo quickly. They are too small and unrepresentative to establish production message-filter quality.

Accuracy is the fraction of all predictions that match the label. Precision asks how many predicted-spam messages are actually spam. Recall asks how many actual spam messages were found. F1 balances precision and recall. The cost of missing spam can differ from the cost of hiding a legitimate message, so accuracy alone cannot choose an operating policy.

4. TF-IDF and sparse matrices

Computers need numbers, so the vectorizer creates one feature position per learned token. TF describes occurrence within a message; IDF reduces the weight of terms common across the training corpus. English stop words are removed, and text is lowercased. The resulting matrix is sparse because a short message uses only a small part of the vocabulary.

> Analogy: imagine a catalogue with a column for every product. One receipt mentions only a few products, so writing thousands of explicit zeroes would waste space. Sparse storage records the useful entries.

The vocabulary and IDF are fitted data. They must be learned inside each training fold. Unseen terms do not create new feature columns during inference. If a message has no known tokens, the API includes a warning; the classifier can still return an intercept-driven result.

5. Logistic regression, probabilities and Pipelines

Logistic regression forms a weighted sum of features and converts it into class probabilities. Its regularisation limits the size of learned weights. A linear classifier is a sensible baseline for sparse text, but the small fixture remains the main limitation here.

For a hypothetical example, an intercept of -1 and a prize feature contributing 1.6 give a score of 0.6. The sigmoid calculation, `1 / (1 + exp(-0.6))`, gives about 0.646. These are teaching numbers, not fitted coefficients from this project.

The scikit-learn Pipeline joins the fitted TF-IDF transform and classifier. `fit` learns; `predict` selects a label; `predict_proba` returns model probabilities. The API's `confidence` is their maximum, rounded to four decimals. It has not been calibrated against representative held-out data.

`joblib.dump` stores the fitted Pipeline. `joblib.load` restores it for serving. Keep versions compatible and only load trusted artifacts. A saved model file is not automatically safe, portable across arbitrary library versions or tracked in a model registry.

6. Cross-validation and leakage

Stratified five-fold validation preserves approximate class proportions. Each run trains on 32 messages and tests on eight. The entire Pipeline is fitted separately each time, so held-out text cannot teach the vectorizer its vocabulary or IDF.

The measured fold accuracies are 87.5%, 87.5%, 62.5%, 87.5% and 62.5%. Their mean is 77.5%; population standard deviation across these folds is 12.25 percentage points. The final artifact is then fitted on all 40 records and reaches 100% training accuracy. Those numbers answer different questions.

> Analogy: practising all the answers can produce a perfect practice score. Testing on questions left out of practice gives a better check, but eight questions still make a fragile examination.

There is no independent real-world holdout, hyperparameter-selection protocol or probability calibration study. The five-fold result should be described as a fixture diagnostic. Fold standard deviation is not a confidence interval.

7. HTTP, FastAPI, Uvicorn and validation

HTTP lets one program send a request and receive a response. FastAPI defines the routes and request contract; Uvicorn runs the application as an ASGI server. Pydantic validates JSON against the `Message` model before prediction logic receives it.

ASGI is the interface through which the server calls the Python application; it is not a separate prediction model.

Request/result	Meaning here
`GET /health` -> 200	Process can answer
`GET /ready` -> 200	Model loaded successfully
`GET /ready` -> 503	Model unavailable
`POST /predict` -> 200	Valid request; classifier result
`POST /predict` -> 422	Request violates the text schema
`POST /predict` -> 503	Model unavailable

The application lifespan handler loads once per process. It preserves fast liveness while readiness reflects the actual model state. A missing artifact requires training and a service restart. There is no automatic model refresh or background retraining.

8. Docker and Compose networking

An image is a packaged filesystem plus runtime instructions; a container is a running instance. The serving Dockerfile installs dependencies before copying source to improve build-cache reuse, trains at build time and starts Uvicorn bound to `0.0.0.0` inside the container.

`EXPOSE 8000` documents the container port. `-p 8767:8000` maps host port 8767 to it. Compose gives services a shared network and names such as `kafka` and `spam-api`. Inside another container, localhost means that container itself.

Kafka listeners bind sockets; advertised listeners tell clients where to reconnect after receiving broker metadata. This Compose stack advertises `kafka:9092`, appropriate for its containers. A host port mapping does not make that advertised hostname valid for every host client. Health-based `depends_on` waits for readiness, rather than merely container creation.

9. Kafka topics, partitions, groups and offsets

A topic is a named log, split into partitions. Each record has a position called its offset. Ordering is per partition. A consumer group distributes partitions among consumers; adding consumers beyond the partition count does not create additional active work for that topic.

The project writes inputs to `messages` and results to `predictions`. A committed offset names the next record to read after restart. `auto_offset_reset=earliest` applies when there is no valid committed offset; it does not force every restart to replay the whole topic.

KRaft lets Kafka's own controllers manage cluster metadata using the Raft consensus protocol, instead of a separate ZooKeeper service. This demo combines controller and broker roles on one node; it does not demonstrate a resilient controller quorum.

> Analogy: a partition is an ordered notebook and an offset is a page number. Committing is moving a bookmark. Moving it before you finish copying the result can make an unfinished page look completed.

The consumer disables background commits, waits on the producer's send future and then commits the exact next input offset. Acknowledgement failure leaves the input uncommitted. A crash between successful output and input commit can still duplicate output, so the guarantee is at least once. Stable event IDs support a future deduplicator, which is not implemented.

`acks=all` waits for the broker's required in-sync acknowledgements: confirmations from replicas considered sufficiently caught up with the leader. With one broker and replication factor one, it still means only one copy. This demonstration has no multi-broker durability test, transactions, dead-letter topic or automatic poison-record recovery.

10. Kubernetes reconciliation and probes

A Pod runs containers; a Deployment manages replaceable replicas; a Service selects matching pods and gives callers a stable network target. The manifests request two API replicas and define CPU/memory requests and limits.

The startup probe allows model loading. Readiness checks `/ready` to decide whether to route traffic. Liveness checks `/health` to decide whether the process should be restarted. These probes ask different questions. The integration test checks two ready replicas and replacement after deleting one pod; that is not a cross-node disaster-recovery benchmark.

11. Terraform, state and LocalStack

Terraform describes desired resources in HCL and uses provider plugins to call APIs. `init` installs providers, `plan` previews changes, `apply` performs them and `destroy` removes the selected resources. State records Terraform's association between configuration and resource identities; it must be protected because it can contain sensitive values.

LocalStack emulates AWS APIs locally. The supplied configuration uses dummy credentials and local endpoints for an S3 bucket and a DynamoDB table. The table uses string `id` as its key and on-demand billing mode. Neither resource is consumed by the serving or streaming application yet.

The acceptance check applies both resources, verifies them through the emulator, checks for an empty second plan and destroys the isolated test resources. It proves an emulated lifecycle, not production AWS behavior or a deployed Kubernetes cluster created by Terraform.

12. Bedrock, interpretation and the limits of the LLM layer

Boto3 calls the Bedrock runtime API using credentials supplied externally. The CLI first requests the classifier's real label and score, then builds the optional LLM request. Credentials and model access are necessary for a live call; mocked SDK tests do not provide that access or prove a live explanation's quality.

The LLM sees the message and prediction, not the classifier's measured feature contributions. Its explanation is therefore a possible interpretation. A faithful attribution exercise would inspect the vectorizer features and model coefficients or use an appropriate attribution method. Asking an LLM "why" does not create that evidence.

Sources: Project implementation | Architecture and evidence | FastAPI lifespan | Kafka producer API | Kubernetes probes | Terraform workflow